Paiflow API quickstart

Updated Oct 12, 2026

Connect your own app to a deployed Paiflow flow without using the hackathon starter. This example shows the complete payment path: your server prepares a payment, the customer signs in Freighter, your server submits it, and your app checks the outcome and payout events.

Choose the right resource

Resource	Use it when you want to…
Paiflow Guide	Set up wallets, build and deploy a flow, then configure and customise the ready-wired starter.
API quickstart	Integrate your own app without the starter, using a complete frontend/backend example.
Full API reference	Look up endpoint schemas, errors, limits, pagination and advanced workflows.
OpenAPI JSON	Import the machine-readable API contract into development tools.
Postman collection	Try API requests with your deployment UUID and token.
AI developer context (`llms.md`)	Give your AI assistant product rules, API details and worked examples.
Starter repository	Get working payment preparation, Freighter signing, submission and event reads without implementing them yourself.
Freighter Testnet setup (video)	Create a wallet, select Testnet and obtain testnet XLM.
Stellar Expert Testnet	Inspect transaction evidence and verify actual payouts.

Follow your participant brief for event rules, preparation permissions, deadlines and submissions.

Starter users can follow the Paiflow Guide without copying this quickstart's `route.js` or `page.js`. This quickstart implements a separate integration and does not use the starter's mode settings.

01 Prepare a deployment and app

> **Dev mode will stay disabled for the duration of the hackathon.** Configure recipients and required values before deploying. Per-payment recipients, **Fill value via API** and **Dev values** are unavailable; standard Pay and Split flows remain available. `PAIFLOW_MODE=team` and `pnpm dev` do not enable flow Dev mode.

Build and deploy a standard **On Receive XLM → percentage Split XLM** flow: 90% to a shop and 10% to a community fund. Keep **Dev mode off** for this example and use two distinct funded Testnet recipient addresses. Wait for **CONFIRMED**, then create a deployment token in **API access** and copy its UUID.

Use a full/team Paiflow account at beta.paiflow.xyz. Sandbox accounts cannot create tokens or use their deployments through the partner API. API access requires an **ACTIVE** deployment and active owner; retiring/retired deployments and revoked/expired tokens are refused.

The example charges **1 XLM**, so expect **0.9 XLM and 0.1 XLM** in payouts. The customer needs Testnet XLM for the payment and fees. One deployment serves all customers with the same configured rules.

This example uses a **Next.js App Router app with JavaScript**. The same server/browser boundary applies to other stacks.

Install development tools

For this JavaScript example, use **Node.js 22 and pnpm 10**. On the Node.js download page, select **22.x** in the version selector and install it for your operating system. Then open a terminal and install pnpm using npm, which comes with Node.js:

```
node --version
npm install --global pnpm@10
pnpm --version
```

Check that the versions begin with `v22.` and `10.` respectively. If both tools are already installed at those versions, skip installation. These tools run this example app; Paiflow's API can also be used from other languages and frameworks.

Create an app and install integration packages

> **Starting a new app:** run the app generator:

```
pnpm create next-app@latest paiflow-api-example
```

When asked whether to use the recommended defaults, choose **No, customize settings**. Select:

- **TypeScript:** No, so the app uses JavaScript and `app/page.js`.
- **React Compiler:** No for this example.
- **`src/` directory:** No, so `app/` is at the project root.
- **App Router:** Yes.
- **Cache Components:** No for this example, if prompted.

Keep the other settings at their defaults. Then enter the new project folder and install the two integration packages:

```
cd paiflow-api-example
pnpm add @stellar/freighter-api server-only
```

> **Using an existing Next.js app:** open a terminal in its project folder and run `pnpm add @stellar/freighter-api server-only`. Skip creating a new project. If your app uses a `src/` directory, put the example files under `src/app/` instead of `app/`.

The packages have separate roles: `@stellar/freighter-api` connects to the browser wallet and requests signatures; `server-only` prevents the backend module from being imported into browser code.

In section 2, create `app/api/payment/route.js` using the server-route code. In section 3, replace the contents of `app/page.js` with the checkout-page code. Keep the existing `app/layout.js` file; it provides the page's required layout.

Create or edit `.env.local` in your project root. Set these server-only values, replace both placeholders with your deployment UUID and token, and restart the dev server if it is running:

```
PAIFLOW_BASE_URL=https://beta.app.paiflow.xyz
PAIFLOW_DEPLOYMENT_ID=YOUR_CONFIRMED_DEPLOYMENT_UUID
PAIFLOW_API_TOKEN=YOUR_DEPLOYMENT_API_TOKEN
```

Never prefix the token with `NEXT_PUBLIC_`, send it to the browser or commit `.env.local`. This independent example has no starter mode setting; you may connect and test before the event as soon as your deployment and API token are ready. No wallet secret keys belong in your app.

02 Add the server route

Create `app/api/payment/route.js`. It exposes only preparation, submission, transaction lookup and payment events for your configured deployment. The browser supplies a public customer address; the server sets the price. Clients cannot choose an upstream URL, deployment or payment amount.

```
import "server-only";

async function upstream(path, options = {}, preparationRequestId = null) {
  const origin = new URL(process.env.PAIFLOW_BASE_URL);
  const id = process.env.PAIFLOW_DEPLOYMENT_ID;
  const token = process.env.PAIFLOW_API_TOKEN;
  if (
    origin.protocol !== "https:" ||
    origin.pathname !== "/" ||
    origin.username ||
    origin.password ||
    origin.search ||
    origin.hash ||
    !/^[a-f0-9]{8}(-[a-f0-9]{4}){3}-[a-f0-9]{12}$/i.test(id || "") ||
    !/^pfk_[a-f0-9]{64}$/.test(token || "")
  ) {
    throw new Error("Invalid server configuration");
  }
  const response = await fetch(new URL(`/api/v1/deployments/${id}/${path}`, origin), {
    ...options,
    headers: {
      Authorization: `Bearer ${token}`,
      "Content-Type": "application/json",
      ...(preparationRequestId ? { "x-preparation-request-id": preparationRequestId } : {}),
    },
    cache: "no-store",
    signal: AbortSignal.timeout(35000),
  });
  const headers = { "Cache-Control": "no-store" };
  for (const name of ["Retry-After", "x-request-id", "x-history-request-id"]) {
    const value = response.headers.get(name);
    if (value) headers[name] = value;
  }
  return Response.json(await response.json(), {
    status: response.status,
    headers,
  });
}

function invalid() {
  return Response.json(
    { error: { code: "VALIDATION", message: "Invalid payment request" } },
    { status: 422 },
  );
}

function unavailable() {
  return Response.json(
    {
      error: {
        code: "UNAVAILABLE",
        message: "Request uncertain. Check the original payment before trying a new one.",
```

```
      },
    },
    { status: 502 },
  );
}

export async function POST(request) {
  try {
    const body = await request.json().catch(() => null);
    if (!body || typeof body !== "object" || Array.isArray(body)) return invalid();
    if (
      body.action === "prepare" &&
      Object.keys(body).length === 2 &&
      typeof body.from === "string" &&
      /^G[A-Z2-7]{55}$/.test(body.from)
    ) {
      return await upstream("execute", {
        method: "POST",
        body: JSON.stringify({ from: body.from, amount: "10000000" }),
      });
    }
    if (
      body.action === "submit" &&
      Object.keys(body).every((key) =>
        ["action", "signedXdr", "preparationRequestId"].includes(key),
      ) &&
      (body.preparationRequestId == null ||
        (typeof body.preparationRequestId === "string" &&
          /^[a-f0-9]{8}(-[a-f0-9]{4}){3}-[a-f0-9]{12}$/i.test(body.preparationRequestId))) &&
      typeof body.signedXdr === "string" &&
      body.signedXdr.length >= 10 &&
      body.signedXdr.length <= 200000
    ) {
      return await upstream(
        "execute/submit",
        {
          method: "POST",
          body: JSON.stringify({ signedXdr: body.signedXdr }),
        },
        body.preparationRequestId,
      );
    }
    return invalid();
  } catch {
    return unavailable();
  }
}

export async function GET(request) {
  try {
    const query = new URL(request.url).searchParams;
    const hash = query.get("txHash") || "";
    if (!/^[a-f0-9]{64}$/i.test(hash)) return invalid();
    if (query.get("kind") === "status") {
      return await upstream(`transactions/${hash}`);
    }
    if (query.get("kind") === "events") {
      return await upstream(`events?txHash=${hash}&limit=100`);
    }
    return invalid();
  } catch {
    return unavailable();
  }
}
```

The API validates the address checksum and transaction contents; the local route also restricts the accepted shape. `amount` is an integer string in the input asset's smallest units: **1 XLM =** `"10000000"`. For a real checkout, calculate the price from your server's order data with exact arithmetic, rather than trusting a browser-supplied total.

The route forwards `x-history-request-id` from preparation. The browser keeps that non-secret ID with the signed envelope and sends it back on every submit/retry; the route forwards it as `x-preparation-request-id`. It is analytics metadata, not wallet authorization. A new preparation gets a new ID. Older deployments without the header still work, but those submissions remain unlinked in analytics.

03 Connect Freighter, sign and submit

Replace `app/page.js` with this browser page. Install/unlock the Freighter extension, select Testnet and use a funded customer account. The payment button requests access to your local app; access granted to Paiflow is separate.

```
"use client";

import { useRef, useState } from "react";
import {
  requestAccess,
  getAddress,
  getNetworkDetails,
  signTransaction,
} from "@stellar/freighter-api";

const TESTNET = "Test SDF Network ; September 2015";

async function api(url, body) {
  const response = await fetch(url, {
    ...(body
      ? {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify(body),
        }
      : {}),
    cache: "no-store",
  });
  const result = await response.json();
  if (!response.ok || result.error) {
    const error = new Error(result.error?.message || `HTTP ${response.status}`);
    error.retryAfter = response.headers.get("Retry-After");
    error.requestId = response.headers.get("x-request-id");
    throw error;
  }
  if (body?.action === "prepare") {
    return { ...result.data, preparationRequestId: response.headers.get("x-history-request-id") };
  }
  return result.data;
}

export default function Checkout() {
  const envelope = useRef(null);
  const working = useRef(false);
  const [busy, setBusy] = useState(false);
  const [uncertain, setUncertain] = useState(false);
  const [result, setResult] = useState(null);
  const [events, setEvents] = useState(null);
  const [message, setMessage] = useState("");

  async function run(action) {
    if (working.current) return;
    working.current = true;
    setBusy(true);
    setMessage("");
    try {
      await action();
    } catch (error) {
      setMessage(
        [
          error.message,
          error.retryAfter && `Wait ${error.retryAfter} before retrying.`,
```

```
        error.requestId && `Request ID: ${error.requestId}`,
      ]
        .filter(Boolean)
        .join(" "),
    );
  } finally {
    working.current = false;
    setBusy(false);
  }
}

async function submit() {
  const data = await api("/api/payment", {
    action: "submit",
    ...envelope.current,
  });
  setResult(data);
  if (data.status === "SUCCESS" || data.status === "FAILED") {
    envelope.current = null;
    setUncertain(false);
  }
}

async function pay() {
  if (envelope.current) throw new Error("Check the existing payment first.");
  const access = await requestAccess();
  if (access.error || !access.address) throw new Error("Allow Freighter access.");
  const from = access.address;
  const network = await getNetworkDetails();
  if (network.error || network.networkPassphrase !== TESTNET) {
    throw new Error("Switch Freighter to Testnet.");
  }
  setResult(null);
  setEvents(null);
  const prepared = await api("/api/payment", { action: "prepare", from });
  const expiry = Date.parse(prepared.expiresAt);
  if (
    prepared.network !== "testnet" ||
    prepared.networkPassphrase !== TESTNET ||
    !Number.isFinite(expiry) ||
    Date.now() >= expiry
  ) {
    throw new Error("Check the preparation network and expiry.");
  }
  const current = await getAddress();
  if (current.error || current.address !== from) throw new Error("Wallet changed. Start again.");
  const signed = await signTransaction(prepared.xdr, {
    networkPassphrase: TESTNET,
    address: from,
  });
  if (signed.error || !signed.signedTxXdr || signed.signerAddress !== from) {
    throw new Error("Payment was not signed by the expected customer.");
  }
  envelope.current = {
    signedXdr: signed.signedTxXdr,
    preparationRequestId: prepared.preparationRequestId,
  };
  setUncertain(true); // Save before submit, including timeout/error cases.
  await submit();
}

async function inspect(kind) {
  const data = await api(`/api/payment?kind=${kind}&txHash=${result.txHash}`);
  if (kind === "events") {
    setEvents(data);
  } else {
    setResult(data);
    if (data.status === "SUCCESS" || data.status === "FAILED") {
      envelope.current = null;
      setUncertain(false);
    }
  }
}

return (
```

```
    <main>
      <h1>Testnet checkout: 1 XLM</h1>
      <p>
        Expected payout: 0.9 XLM to the shop and 0.1 XLM to the community fund. Fees are additional.
      </p>
      <button disabled={busy || uncertain} onClick={() => run(pay)}>
        Pay 1 XLM
      </button>
      <button disabled={busy || !uncertain} onClick={() => run(submit)}>
        Check again
      </button>
      <button disabled={busy || !result?.txHash} onClick={() => run(() => inspect("status"))}>
        Look up status
      </button>
      <button disabled={busy || !result?.txHash} onClick={() => run(() => inspect("events"))}>
        Read payout events
      </button>
      {busy && <p>Waiting for the wallet or network…</p>}
      {message && <p role="alert">{message}</p>}
      {result && <pre>{JSON.stringify(result, null, 2)}</pre>}
      {events && <pre>{JSON.stringify(events, null, 2)}</pre>}
    </main>
  );
}
```

Run `pnpm dev` and open `http://localhost:3000`. Review the asset, amount, network and account in Freighter before confirming. Preparation lasts approximately **180 seconds**; have the wallet ready. Signing stays inside Freighter; the bearer token stays on your server. The browser receives only unsigned preparation data and payment results.

04 Verify the payment and payouts

Select **Pay 1 XLM**, sign in Freighter, and inspect the returned `status` and `txHash`. HTTP 200 alone is not payment confirmation.

Result	What to do
`SUCCESS`	Payment confirmed. Select Read payout events and verify the expected two recipients and amounts.
`FAILED`	Submission rejected or transaction failed. Read the error and check transaction status before preparing another payment.
`PENDING` or an uncertain submit error	Keep the tab open. Check again submits the same signed envelope; it does not create a second payment.

Look up status checks `GET /api/v1/deployments/{id}/transactions/{txHash}` without resubmitting. It may also return `REJECTED` (submission refusal, not on-chain failure) or `UNKNOWN` (insufficient evidence). Neither `UNKNOWN`, a `404`, nor a timeout proves failure or authorises a replacement payment. Keep checking the original transaction; create a replacement only after confirmed failure or verified expiry without inclusion on-chain.

Read payout events calls `GET /api/v1/deployments/{id}/events?txHash=…&limit=100`. Check `items`, `nextCursor` and `hasMore`. For this two-recipient example, verify **0.9 XLM to the shop and 0.1 XLM to the community fund**, using the event data and the transaction on Stellar Expert Testnet. A percentage Split event can describe shares rather than literal transfer amounts; use transaction evidence to verify the actual transfers.

A delayed feed is not proof of failure. This example reads one event page on demand. For continuous feeds, follow `nextCursor` while `hasMore` is true, deduplicate by `eventId`, associate activity by `txHash`, and back off between polls. The API reference explains event shapes and pagination.

05 Adapt it to your app

Keep the payment sequence intact while replacing the fixed price and UI with your idea. Use server-side order totals and exact integer amounts. Your deployment sets its shared recipients and rules; the API executes that deployment, rather than creating a flow for every customer.

The example preserves signed envelopes only in tab memory. Reloading loses that retry state. Before starting another payment after a reload, check the original transaction through your wallet/explorer. A real app needs a durable payment record with its transaction hash, clear pending states, and its own authentication and order-to-transaction checks. Avoid logging signed envelopes or credentials.

Changing settings or closing the page does not cancel a submitted transaction. A token authenticates your backend to Paiflow; it does not replace customer signing or your app's own authorisation. This sample does not implement subscriptions, schedules, webhooks, early release or a continuous feed.

For other flow types, first inspect `GET /api/v1/deployments/{id}/capabilities` with the same bearer token. Supported paths still require live checks for balances, consent and configuration. Use the reference for those integrations instead of treating them as ordinary On Receive payments.

06 Explore other API workflows

The checkout above uses `execute` for an On Receive deployment. Other deployed flows have different funding and wallet requirements. Ask `GET /api/v1/deployments/{id}/capabilities` from your server first. Its operation statuses describe possible integrations; read the corresponding state or preparation endpoint to check current eligibility. Keep the bearer token on your server and have the relevant wallet sign each prepared envelope.

Read contract balances

`GET /api/v1/deployments/{id}/balances` returns current contract token holdings, read timestamps and freshness status for supported deployed nodes. Check `operations.balances` in capabilities. An incomplete or stale result is not a zero balance, and Schedule contract holdings are not the same as vested payout availability. Use the balance reference for the response shape and limits.

Preview one deployed Swap

The source endpoint `GET /api/v1/deployments/{id}/swap-quote?amountStroops=10000000` uses that deployment's bearer token. Hosted quote-versus-execution testnet evidence is pending; check the hosted OpenAPI before relying on availability. Discover candidate nodes through `operations.swapQuote`; pass `nodeId` if multiple Swaps exist. Only amount and node selection are accepted, with assets/router/slippage read from the deployed contract. This is an estimate of the input reaching that node, never a whole-flow prediction or guaranteed payout. Read timestamps describe successive live simulations; later preparation/execution can differ. See the API reference for exact stroop output, estimated minimum, eligibility and 20/min deployment quote budget.

Fund a Schedule or Webhook pool

Check `operations.funding`, then use GET/POST `/api/v1/deployments/{id}/funding` to discover the live asset and prepare a top-up. The funding wallet signs the returned XDR; your server calls `/funding/submit` and recovers uncertain attempts through `/funding/reconcile`. Funding adds funds only; it does not execute payouts or callbacks. One unresolved intent per node blocks a replacement until confirmed finality or ledger-proven expiry. Webhook callbacks separately require `x-webhook-secret`. Follow the funding and recovery example. Hosted testnet verification remains pending.

Read and control a Schedule

For Schedule integrations, discover `scheduleState` and `scheduleControls`, then read `/api/v1/deployments/{id}/schedule/state`. Controller-wallet pause/resume/retrieve use action-specific preparation, `/submit` and `/reconcile`. The deployment token grants API access; the controller wallet signs the action. Keep the token on your server while the browser wallet signs. Funding is separate. Follow the Schedule wallet and recovery example. Hosted testnet verification remains pending.

Read and control a Subscription

`GET /api/v1/deployments/{id}/subscription/state` reads the configured subscriber, current allowance and cancellation state for an eligible deployment. Check `subscriptionState`, `subscriptionAllowance`, `subscriptionCancel` and `subscriptionReactivate` in capabilities. The configured subscriber signs allowance approval or a prepared cancel/reactivate transaction; the deployment token alone cannot give consent. Cancellation revokes allowance. Reactivation grants no new allowance and does not charge immediately. Use the Subscription control reference for prepare, submit, transaction lookup and reconcile. Hosted testnet verification remains pending.

07 References

- Full API reference — schemas, errors, limits, pagination and advanced workflows.
- OpenAPI JSON — the machine-readable contract; check the live `/api/v1/openapi.json` for the deployed version.
- Postman collection — set `baseUrl`, deployment UUID and token for your instance.
- Freighter signing API — browser wallet signature interface.
- Paiflow Guide — wallet/flow setup and the ready-wired starter path.

When requesting support, include the deployment UUID, transaction hash, error message and request ID when available. Keep tokens, wallet secrets and signed envelopes out of support messages.